Course Name: Advanced Programming (OOP) [PCCCS495]

Project Title: NivoraStay – Intelligent Hotel Reservation System

Student Name: Swastika Das

Roll Number: 31

Instructor Name: Soumadip Biswas

Department: Information Technology

Spring 2026

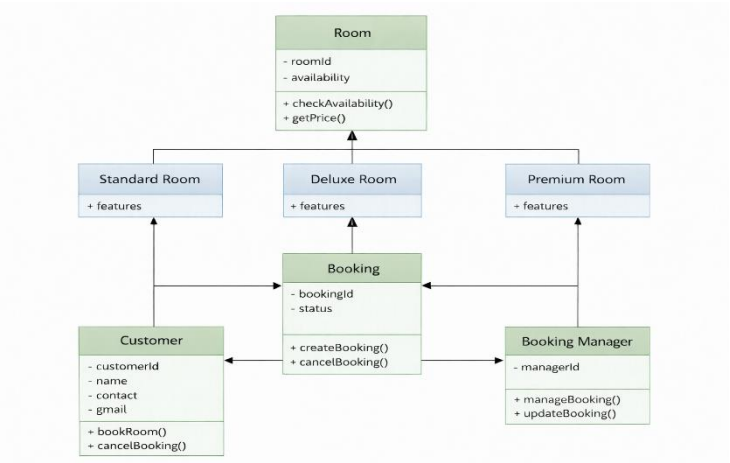
1. Problem Statement:

Many small and medium hotels still rely on manual booking systems or simple spreadsheets to manage reservations. These methods often lead to issues such as double bookings, difficulty tracking room availability, and inefficient customer record management. As a result, hotel staff may struggle to confirm reservations quickly and maintain accurate data.

The NivoraStay-Intelligent Hotel Reservation System aims to solve these problems by providing a structured digital solution using object-oriented design. It models rooms, bookings, and customers as separate objects and uses a centralised BookingManager to handle reservation processing and availability validation, improving efficiency and reducing errors.

2. **System Design:** The BookingManager acts as the central controller coordinating all operations between classes.

- UML Class Diagram-



- **Description of Major Classes**- The system consists of four main classes: Room, Customer, Booking, and BookingManager.

Room: The Room class represents hotel rooms and contains basic details such as room number and type. It serves as a base class for different room categories like StandardRoom, DeluxeRoom, and PremiumRoom.

Customer: The Customer class stores customer information including ID, name, phone number, and email address. It helps in managing and identifying customers in the system.

Booking: The Booking class represents a reservation. It links a customer with a room using a unique booking ID. This class is responsible for storing booking details.

BookingManager: The Booking Manager class acts as the central controller of the system. It manages all operations such as adding customers, booking rooms, cancelling bookings, and checking room availability. It uses collections like ArrayList and HashMap for efficient data handling.

- **Package Structure-** All classes in the project are organised within the src directory. The project follows a simple structure where each class is placed in the same package for ease of implementation and compilation.

```
src/
 ├── Room.java
 ├── Customer.java
 ├── Booking.java
 ├── BookingManager.java
 └── Main.java
```

The src folder contains all the main Java classes including Room, Customer, Booking, BookingManager, and Main. This structure ensures that the code is easy to manage, understand, and execute.

Since the project is designed for simplicity, no separate packages are created, and all classes are kept together in a single package.

- **Interaction Flow-** The interaction flow of the system describes how the user interacts with the application and how the system processes the request.

1. The user starts the application through the Main class.

2. The system displays a menu and takes input from the user.

3. Based on the user's choice, the Main class sends a request to the BookingManager.

4. The BookingManager processes the request and interacts with the required classes such as Room, Customer, and Booking.

5. If a booking is made, the Booking class links the selected room with the customer.

6. The system performs validation such as checking room availability and preventing double booking.

7. The result of the operation is displayed to the user.

This flow ensures proper communication between different components of the system and maintains a clear separation between user interface and business logic.

3. OOP Concepts Demonstrated:

I. **Encapsulation** - ***Where it is used:*** Encapsulation is used in the Customer class where variables such as id, name,phone, and email are declared as private.

Why it is used:
It protects data and allows controlled access through methods.

***Code Snippet*:**

```
private int id;
public int getId() {
    return id;
}
```

Explanation:

The private variables prevent direct access, while getter methods provide controlled access, ensuring data security.

II. Inheritance- ***Where it is used:***

Inheritance is used in the Room class hierarchy where StandardRoom, DeluxeRoom, and PremiumRoom extend the base Room class.

Why it is used:

It helps in code reuse and allows creation of specialized room types without rewriting common properties.

Code Snippet:

```
class StandardRoom extends Room {

    public StandardRoom(int roomNumber) {

        super(roomNumber, "Standard");    }}
```

Explanation: The subclass StandardRoom inherits properties and methods from the Room class. This reduces redundancy and improves maintainability.

III. Polymorphism- ***Where it is used*:** Polymorphism is used when a reference of type Room is used to store objects of different room types.

Why it is used: It allows flexibility by enabling a single reference to represent different types of objects.

Code Snippet:

```
Room room;
room = new DeluxeRoom(102);
```

Explanation:The variable room is of type Room but refers to a DeluxeRoom object. This demonstrates runtime polymorphism.

IV. Use of Collections- ***Where it is used:***

Collections are used in BookingManager to store rooms, customers, and bookings using ArrayList and HashMap.

Why it is used:

They provide efficient and dynamic storage and allow easy retrieval and management of data.

Code Snippet:

```
ArrayList<Booking> bookings = new ArrayList<>();
HashMap<Integer, Customer> customers = new HashMap<>();
```

Explanation: ArrayList allows dynamic storage of bookings, while HashMap

enables fast lookup of customers using their ID.

4. Implementation Highlights:

A. ***Booking Validation (Prevent Double Booking)-***

Code Snippet:

```
if (!isRoomAvailable(roomNumber)) {
    System.out.println("Room already booked!");
    return;
}
```

Justification:

This feature ensures that a room cannot be booked more than once at the same time. It improves system reliability and prevents conflicts in reservations.

B. ***Customer Storage using HashMap-***

Code Snippet:

```
HashMap<Integer, Customer> customers = new HashMap<>();
```

Justification:

Using HashMap allows fast retrieval of customer data using unique IDs. This improves efficiency in searching and managing customer records.

C. ***Booking Storage using ArrayList-***

Code Snippet:

```
ArrayList<Booking> bookings = new ArrayList<>();
```

Justification:

ArrayList provides dynamic storage for bookings, allowing easy addition and removal of booking records.

D. ***Dynamic Room Creation using Inheritance-***

Code Snippet:

```
Room room = new DeluxeRoom(roomNumber);
```

Justification:

This demonstrates dynamic object creation using inheritance. It allows flexibility in handling different room types using a common reference.

5. Testing & Error Handling:

A. Testing & Error Handling-The system was tested using multiple test cases to ensure correct functionality. These tests confirm both functional correctness and system robustness.

- Adding a new customer with valid details.
- Viewing available rooms.
- Booking a room with valid customer ID.
- Updating customer details.
- Canceling a booking using booking ID.

These test cases confirm that the system performs expected operations correctly.

B. ***Edge Cases Handled***-The system handles various edge cases to improve reliability.

- Attempting to book a room that is already booked.
- Entering an invalid customer ID while booking.
- Trying to update a customer that does not exist.
- Booking without adding any rooms or customers.

Handling these cases ensures the system behaves correctly under unexpected conditions.

C. ***Failure Scenarios***-The system includes basic error handling to manage failures.

- Displays a message when a booking is not found.
- Prevents booking if the room is unavailable.
- Handles invalid user input using validation checks.
- Ensures system does not crash due to incorrect inputs.

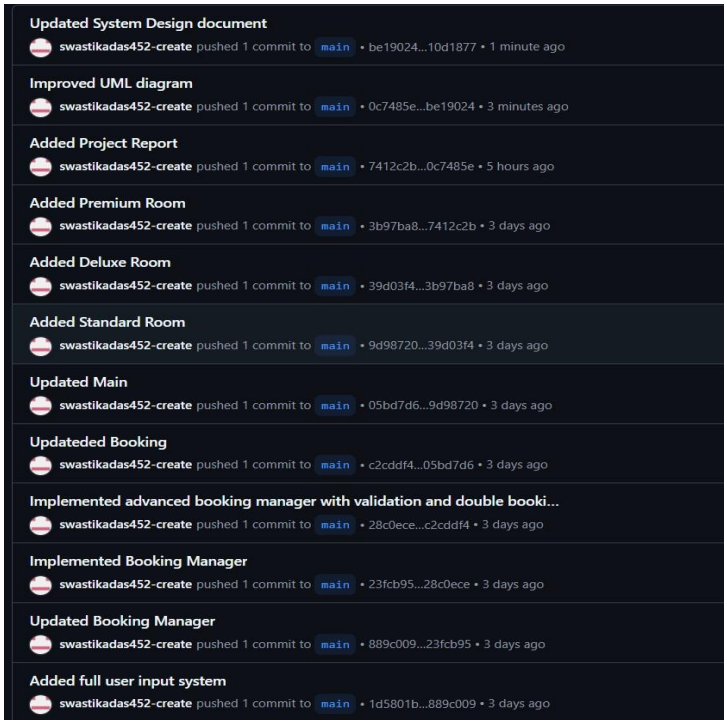
These mechanisms improve system stability and user experience**.**

6. Git Workflow:

The project was developed using proper Git version control practices with multiple meaningful commits.

Each feature of the system was implemented incrementally. Separate commits were made for adding classes such as Room, Customer, Booking, and Booking Manager. Additional commits were made for implementing features like booking validation, menu-driven interaction, and error handling.

The commit messages clearly describe the changes made, ensuring readability and traceability. The development process was carried out in stages rather than a single bulk upload, demonstrating continuous progress.

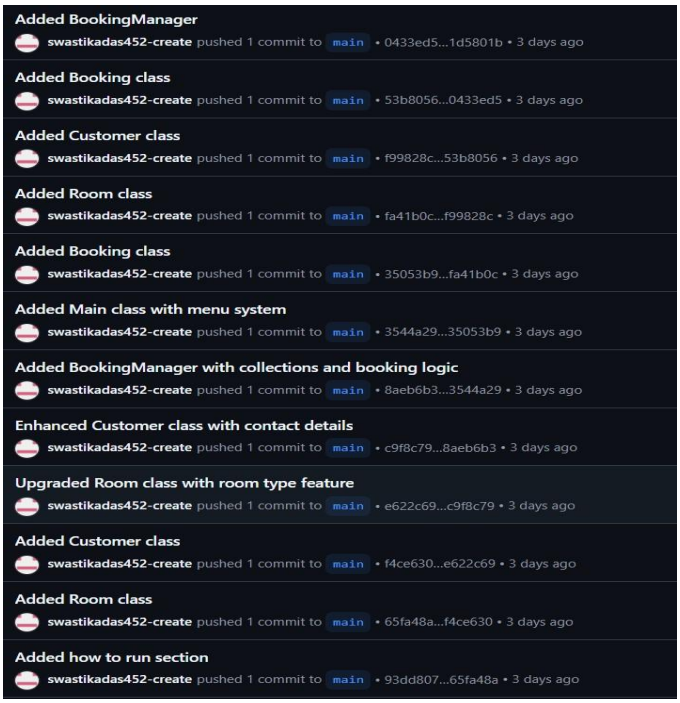


(A screenshot of the commit history is attached)

This workflow helps in tracking changes, managing versions effectively, and maintaining a clean and organized repository.

Examples of commit messages:

- Added Room class implementation
- Added Booking class
- Implemented BookingManager
- Added menu-driven Main class
- Improved customer data handling
- Added validation for double booking
- Updated README with instruction.



(A screenshot of the commit history is attached)

7. Conclusion & Future Scope:

The NivoraStay - Intelligent Hotel Reservation System successfully demonstrates the application of object-oriented programming concepts in solving real-world problems. The system provides an efficient way to manage hotel operations such as room booking, customer data management, and reservation handling.

Key features such as prevention of double booking, use of collections for data storage, and modular class design improve the overall reliability and performance of the system. The use of concepts like encapsulation, inheritance, and polymorphism makes the system structured and maintainable.

Future improvements can include the development of a graphical user interface (GUI) for better user interaction, integration with a database for persistent data storage, and addition of advanced features such as online payment and room recommendation systems.

Overall, the project provides a scalable and efficient solution for hotel reservation management. This project also demonstrates practical application of OOP in real-world scenarios. The project can be further enhanced to support multi-user access and real-time updates.

8. Appendix -

A: Initial Project Proposal:

(Original proposal submitted for project approval)

NivoraStay – Intelligent Hotel Reservation System

Problem Statement:

Many small and medium hotels still rely on manual booking methods or simple spreadsheets to manage reservations.

These methods often lead to problems such as double bookings, difficulty tracking room availability, and inefficient management of customer records. When booking information is poorly organized, hotel staff may struggle to confirm reservations quickly or update room status accurately, which can negatively affect both operations and guest experience.

The NivoraStay - Intelligent Hotel Reservation System aims to solve these issues by providing a structured digital solution. The system models hotel operations using object-oriented design, where rooms, bookings, and customers are represented as separate objects with clearly defined responsibilities. A centralized BookingManager coordinates reservation processing and availability validation. This approach improves efficiency, reduces errors, and demonstrates modular and scalable software design.

Target Users:

The primary users of the system are hotel administrators and reception staff who manage daily reservations and maintain guest records. The system helps them track bookings, check room availability, and keep reservation data organized efficiently. Hotel managers or owners can also use the system to review booking activities and monitor room occupancy, helping them understand hotel operations and manage resources more effectively.

Core Features:

• Room availability tracking
• Room booking and reservation management
• Booking modification and cancellation
• Customer record management
• Booking validation to prevent double reservations
• Persistent storage for reservation data
• Exception handling for invalid booking requests

OOP Concepts to be Used:

Abstraction: Base classes define shared behaviour for system entities.
Inheritance: Customer and Admin classes inherit from a common user class.
Polymorphism: Booking operations may override validation or confirmation behaviour.
Encapsulation: Room, booking, and customer data are protected within classes.
Collections: Lists or maps store rooms and reservation records dynamically.
Exception Handling: The system handles invalid input or booking conflicts.
Design Pattern: The Booking Manager follows the Singleton pattern to ensure that only one instance manages reservations.

System Architecture:

The system follows a layered object-oriented architecture to maintain clear separation of responsibilities. The Room and Booking classes represent the core entities of the system, while the Customer class manages guest information. A central Booking Manager coordinates reservation logic and availability validation.

Finally, a persistence layer manages data storage and retrieval so booking records can be saved and accessed when needed.

Architecture Flow:
Customer / Admin → Booking Manager → Room & Booking Objects → Persistence Layer (Storage)

Appendix B: Review Approval Mail:

Outlook

PCCCS495 Proposal Review – APPROVED_MINOR

From soumadip.biswas@ieminternational.ai <soumadip.biswas@ieminternational.ai>
Date Tue 3/10/2026 9:03 AM
To Swastika Das <Swastika.Das2024.2@iem.edu.in>

Dear Swastika Das,

Project: NivoraStay – Intelligent Hotel Reservation System
Decision: APPROVED_MINOR
Score: 92

Strengths:
The proposal clearly articulates a practical problem in hotel management and proposes a robust, object-oriented solution. The problem statement is well-defined, and the proposed solution effectively maps real-world entities to distinct objects with clear responsibilities. The explicit mention of core OOP concepts (Abstraction, Inheritance, Polymorphism, Encapsulation) along with practical considerations like Exception Handling and Collections demonstrates a strong foundational understanding. The layered architecture is logical and promotes separation of concerns, which is excellent for a university project. The inclusion of persistent storage and booking validation features adds significant value and realism.

Improvements:
While the proposal lists key OOP concepts, it could be strengthened by providing more concrete examples of how Inheritance and Polymorphism will be applied within the system (e.g., different types of rooms inheriting from a base 'Room' class, or various booking statuses dictating polymorphic behavior). Similarly, elaborating on the specific use cases for Java Collections beyond simply 'storing data' (e.g., using a Map for efficient customer lookup, or a List for managing room inventory) would be beneficial. A brief mention of the intended user interface (e.g., console-based, basic GUI) would also help scope the project further.

Regards,
Course Instructor

(The copy of Proposal Review - APPROVED_MINOR is attached)